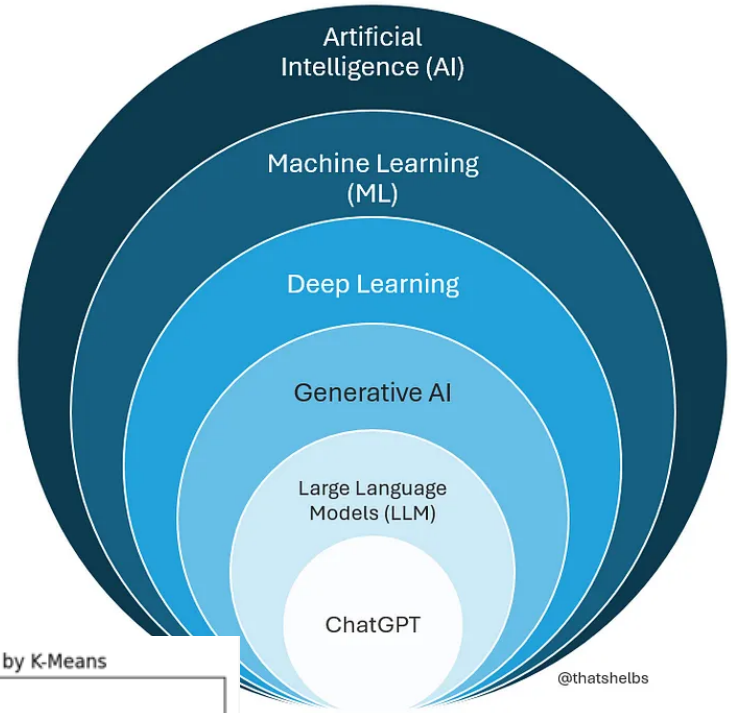
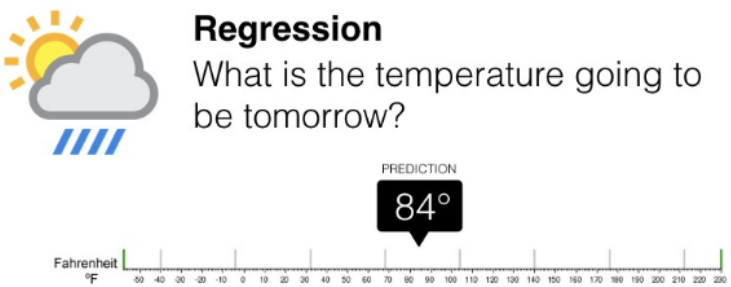
Python 200
Week 2: Intro to Machine Learning

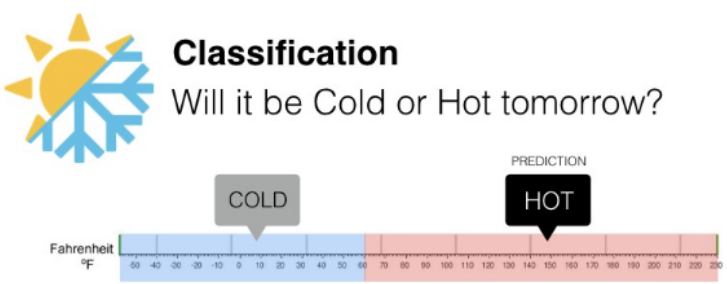
CODE THE DREAM

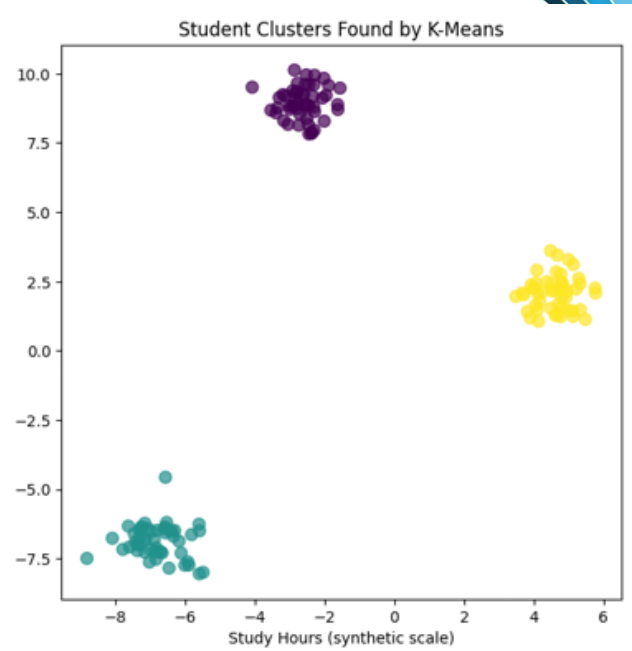
What is Machine Learning?

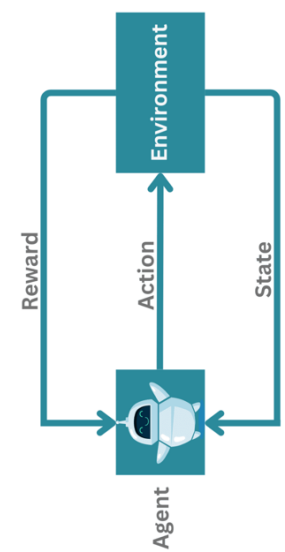
- Learning from examples, not from explicit rules
- AI > Machine Learning > Deep Learning > LLMs
- Supervised Learning: labeled data, predict an output
  - Regression: continuous output (price, temperature)
  - Classification: category output (spam, fraud, churn)
- Unsupervised Learning: no labels, find structure
  - Clustering: group similar examples (K-Means)
- Reinforcement Learning: learn by trail and error.
- This week: supervised regression











The scikit-learn API

```
# 1. Create
model = LinearRegression()

# 2. Fit (learn from training data)
model.fit(X_train, y_train)

# 3. Predict
y_pred = model.predict(X_test)

# 4. Evaluate
model.score(X_test, y_test)
```

- Same pattern for every model:
  - LinearRegression, KMeans, SVM...
- X must always be 2D

```
shape = (n_samples, n_features)
500 houses, 1 feature  ->
   (500, 1)
500 houses, 3 features ->
   (500, 3)
```

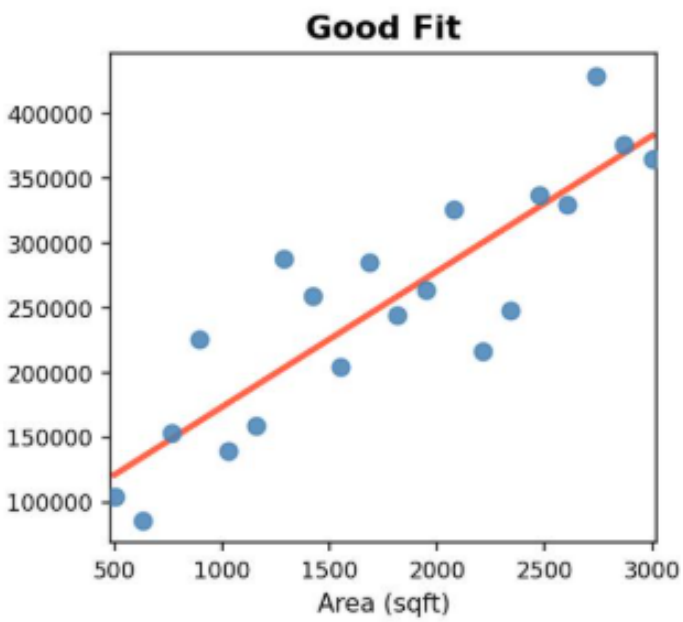
- y is 1D (n_samples,)
- Use .reshape(-1, 1) for 1-feature X

Simple Linear Regression

```
X = df[["sqft"]]   # shape: (500, 1)
y = df["price"]

model = LinearRegression()
model.fit(X_train, y_train)

print(model.coef_[0])   # ~$107/sqft
print(model.intercept_)
```



- price = slope x sqft + intercept
  - Slope: change in price per sqft
  - Here: ~$107 per additional sqft
  - Intercept: price when sqft = 0
- The best-fit line minimizes prediction
  - error across all training examples

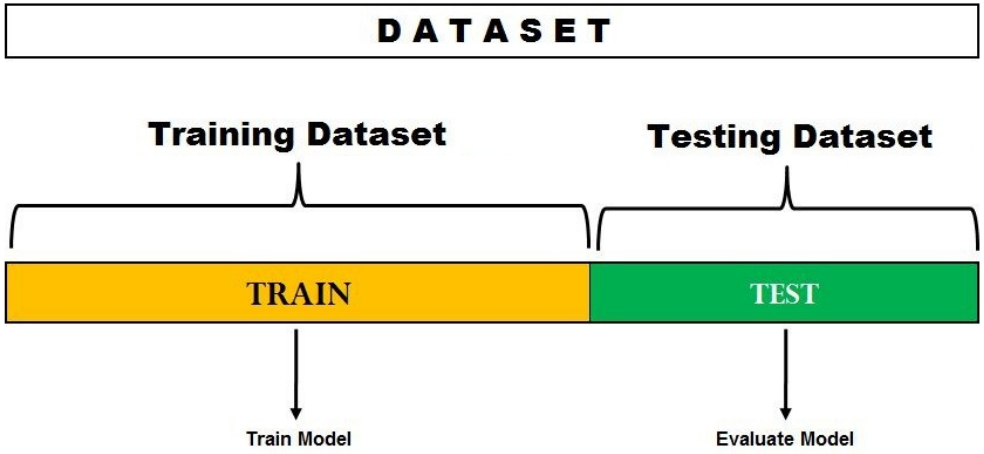
Train/Test Split: Evaluating Generalization

```
from sklearn.model_selection import \
    train_test_split

X_tr, X_te, y_tr, y_te =
    train_test_split(
        X, y,
        test_size=0.2,
        random_state=42)

# 400 houses to train on
# 100 houses to evaluate with
```

- A model must work on data it has never seen
- Train set: model learns from these examples
- Test set: simulates the real world
  - Never let the model train on test data
- 80/20 is a common starting split
- random_state fixes the shuffle
  - results are reproducible across runs



Evaluating the Model: RMSE and R^2

```
y_pred = model.predict(X_te)

rmse = np.sqrt(mean_squared_error(
    y_te, y_pred))
# ~$42,000

r2 = model.score(X_te, y_te)
# ~0.75
```

- RMSE: average error in original units
  - ~$42k per house -- good or bad?
  - Depends on price range and use case
- R^2: how much better than the mean
  - 0 = no better than predicting the mean
  - 1 = perfect predictions
  - negative = worse than the mean (!)
- For simple regression: $R^2 = r^2$
  - r = Pearson correlation from Week 1
- Always evaluate on the TEST set

Overfitting: Memorizing vs. Learning

- Compare train R^2 and test R^2 -- the gap is the warning sign

- Linear model (healthy):

  `Train R² = 0.66  Test R² = 0.67  gap ≈ 0`

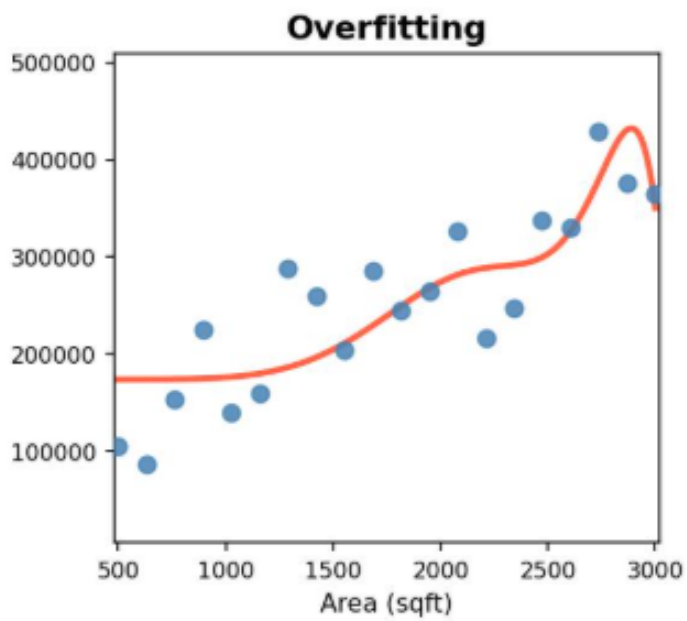
  – Learns the real trend, generalizes well

- Degree-10 polynomial (overfit):

  `Train R² = 0.76  Test R² = -2.16  huge gap!`

  – Memorized training noise, fails on new data

- Linear models are naturally resistant -- few parameters, less flex



Multiple Regression: Adding Features

```
# Add distance as second feature
X = df[["sqft", "distance"]]
# shape: (500, 2)

model = LinearRegression()
model.fit(X_tr, y_tr)

print(model.coef_)
# [107.3,  -2498.1]

model.score(X_te, y_te)  # 0.89
```

- Identical code -- scikit-learn handles geometry
- sqft: +$107 per sqft
- distance: -$2,498 per mile from city
- Each coefficient: effect of that feature
  - while holding all others constant
- R^2: 0.75 → 0.89 -- distance earns its place

Binary Features

```
# Add is_new  (1=new build, 0=older)
X = df[["sqft", "distance", "is_new"]]

model.fit(X_tr, y_tr)

print(model.coef_)
# [95.4, -2499.0, 27523.4]

model.score(X_te, y_te)   # 0.94
```

- Binary features: encode yes/no as 1/0
- Coefficient = effect of switching 0 → 1
  - New home premium: ~$27,500
- R^2: 0.89 → 0.94 -- feature earns its place
- Test R^2 is the judge:
  - improves = feature adds real signal
  - flat or drops = adding noise

Key Takeaways

- scikit-learn: create → fit → predict (every model)
- X shape: always 2D (n_samples, n_features)
- Metrics: RMSE in original units, R^2 on a 0-1 scale
- Train/test split: always evaluate on held-out data
- Multiple regression: same code, more features
- Overfitting: big train-test R^2 gap is the warning sign